

Predicting Content Decline Before It Shows Up in the Numbers

A calibrated, client-grouped model for prioritizing which pages a content team should review first — and an honest account of how far that model can actually be trusted.

Author: Abhay lokhande | Lane: Refresh / Content Opportunity Scoring | Repo: < <https://github.com/space-0d/flyrank> > |
Date: August 29, 2026

PR-AUC vs. base rate	Lift (59.3% vs 43.1% base, n=2,458/18,784)	Share of pages flagged
0.524 vs 0.431	1.47×	13.1%

0 ABSTRACT

Can a content team know which pages are likely to decline before that decline already shows up in the traffic dashboard everyone already watches? This report builds on the FlyRank ML Internship dataset — 71,074 anonymized content-performance observations, tracked across pseudonymous clients with 90-day and 30-day trailing windows. A calibrated Random Forest classifier was trained on 23 pre-cutoff features and validated on a client-grouped hold-out split of 18,784 pages, so that no client's pages influence both training and evaluation. Against a 43.1% base decline rate, the final model reaches a PR-AUC of 0.524 and, at its chosen operating threshold, flags the top 13.1% of pages with 59.3% precision — a 1.47× lift over picking pages at random. The output is a ranked, probability-scored review queue intended to help a content editor decide where to look first, not a causal claim about what fixes decline, and not a replacement for that editor's judgment.

1 INTRODUCTION / PROBLEM

Where the manual review breaks down

The unit of analysis in this project is a single content page, identified only by an anonymized `content_hash_id` within an anonymized `client_hash_id`. A content team managing a large page inventory cannot manually re-review every page every month to check whether it is starting to lose ground — there are simply too many pages, and decline is gradual enough that it is easy to miss until it is already visible in a monthly report.

The decision this work supports is narrow and deliberately modest: given everything known about a page before a cutoff date, rank pages by how likely they are to decline afterward, and hand a content editor a short, prioritized list to review first. The action a human takes from the output is inspection, not automated change — a flagged page gets looked at; nothing gets rewritten by the model itself.

The cost of a wrong call is asymmetric and worth naming plainly. A false positive costs an editor a few minutes reviewing a page that turns out to be fine. A false negative means a genuinely declining page waits until the next manual pass, or until it shows up unprompted in a client's own performance report.

Neither failure is catastrophic, which is precisely why a probabilistic, ranked queue — rather than a hard automated decision — is the right shape for this output.

Data and machine learning help here specifically because the signals that matter — recent performance, staleness, search demand, and competitive pressure — do not move independently, and no simple rule of thumb weighs all of them at once as reliably as a model trained on the joint pattern across thousands of pages.

2 DATA

What went in, and what was deliberately left out

This analysis uses the anonymized content-performance dataset prepared for the FlyRank ML Internship. The final modeling dataset contains 71,074 observations, each identified only by pseudonymous `client_hash_id` and `content_hash_id` values — no client names, domains, or URLs appear anywhere in this report or in the underlying notebooks.

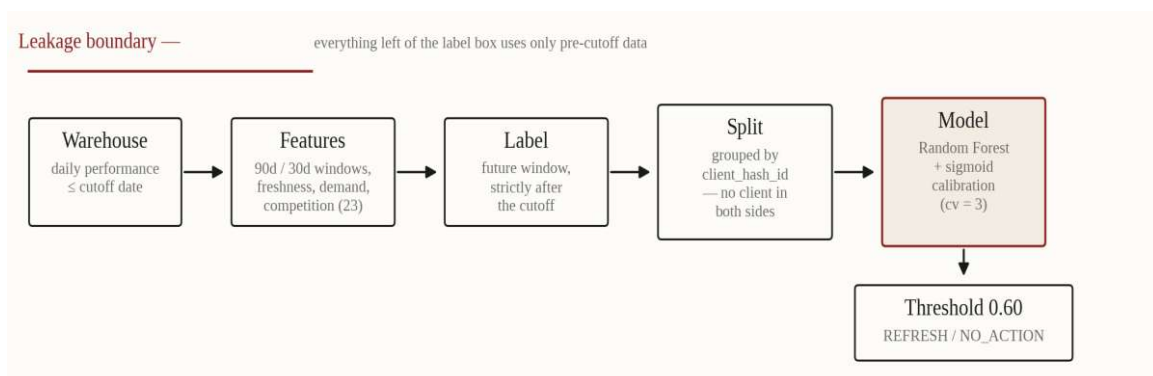
Two time windows anchor every feature: a 90-day historical window and a 30-day recent window, both measured relative to a fixed cutoff date. The outcome label is built from a separate window that falls strictly after that cutoff, so that the model is always being asked to predict something it could not have observed at prediction time.

Excluded on purpose

- **Label-derived fields** — any pre-computed trend or direction indicator that already encodes the outcome was excluded from the feature set entirely, and its removal was verified twice (see §4, Leakage checks).
- **Raw client identity** — `client_hash_id` is used only to group the train/test split, never as a model input, so the model cannot learn "which client" as a shortcut for "which outcome."
- **Anything client-identifying** — confirmed absent from every notebook and from this report.

3 METHODOLOGY

Assumptions, features, label, baseline, and validation



Every feature is built from data at or before the cutoff; the label alone is drawn from a strictly later window. The split is grouped by client so no client crosses between train and test.

Figure 1. The leakage boundary is a design choice, not an afterthought — it separates what the model can see from what it is asked to predict.

Assumptions

This work assumes that recent performance carries early-warning information, that content freshness plausibly influences future performance, and that search demand and competitive pressure provide useful context for prioritization. These are modeling assumptions used to justify feature choices — they are not causal claims, and none of them are tested as such here.

Features and label

The final model uses 23 raw features (32 after encoding), spanning four groups: historical performance (impressions, clicks, average position, CTR — both 90-day and last-30-day), content and freshness (age, days since update, word and character count, content type), search and competition (search volume, competition level, primary intent), and authority (backlinks). The target is a binary future-decline flag, defined from the post-cutoff window described in §2, with a 43.05% positive rate in the modeling data.

Baseline

Stated plainly: the baseline used for comparison in this report is an earlier, 18-feature version of the same Random Forest architecture — not an independent, non-model decision rule (for example, "flag every page untouched for 90+ days"). This is a fair comparison for measuring whether the five additional engineered features improved the model, but it is a narrower claim than "this model beats simple heuristics." No such heuristic was formally scored against the same split, and that gap is treated as an open limitation rather than glossed over.

Validation design

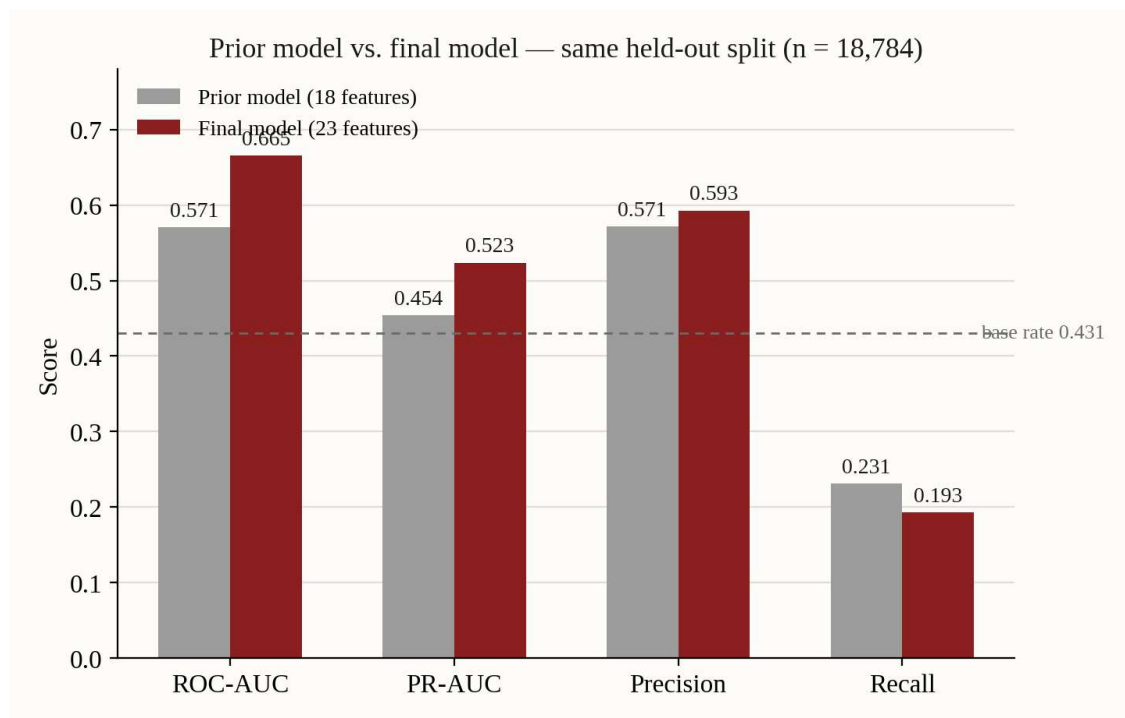
The data was split using a GroupShuffleSplit grouped by `client_hash_id`, holding out 25% of pages for testing. Grouping matters because pages from the same client are correlated — a random row-level split would let the model see other pages from a test client during training, inflating apparent performance. No client appears on both sides of the split.

Leakage checks

Before training, the feature set was audited twice to confirm that no post-cutoff or label-derived column reached the model — including the trend/direction fields excluded in §2 and any feature whose 90-day window could overlap the label's future window. Both audits passed clean.

4 RESULTS

Model vs. prior model, same split



Both models evaluated on the identical held-out test set of 18,784 pages.

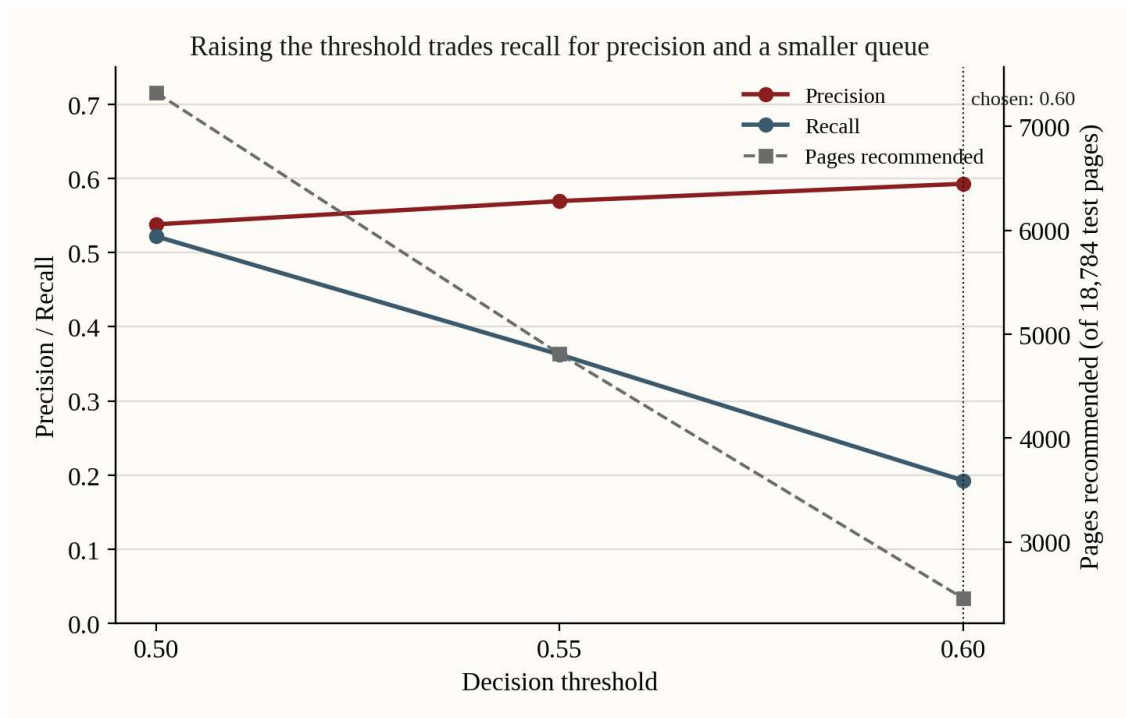
Figure 2. The five added features moved ROC-AUC and PR-AUC up meaningfully, but recall fell — the model became more selective, not simply "better" in every direction.

Table 1. Full comparison, same test split, same 0.60 operating threshold

Metric	Prior model	Final model	Change
Raw features	18	23	+5
ROC-AUC	0.5706	0.6655	+0.095
PR-AUC	0.4540	0.5235	+0.070
Precision @ 0.60	57.13%	59.28%	+2.15 pp
Recall @ 0.60	23.07%	19.29%	−3.78 pp
F1	—	0.2911	—
Pages recommended	3,051	2,458	−593
Lift over base rate	1.421×	1.474×	+0.053×

Base decline rate across the test set: 43.05% — reported here explicitly so that the precision numbers above are read against the right floor, not against zero.

Read together, the improved model should be described as a more selective prioritization model, not as a model that catches every declining page. It trades some recall for higher confidence per recommendation — a defensible choice when the review queue itself is the scarce resource, but a choice worth stating out loud rather than burying in a single headline metric.



Three thresholds tested; 0.60 chosen as the operating point.

Figure 3. Lowering the threshold to 0.50 nearly triples the recommended queue and roughly triples recall, at a real cost to precision — the right threshold depends on how much review capacity the editor team actually has.

5 LIMITATIONS & HONEST FRAMING

What this work cannot claim

1. This does not prove causality

A REFRESH prediction is a measured decline-risk signal, not evidence that refreshing the page will improve its future performance. No experiment (such as an A/B test on actual refresh actions) was run, and none of the numbers in this report should be read as a causal effect.

2. This does not detect every decline

At the operating threshold, recall is 19.29% — meaning roughly four out of five genuinely declining pages in the test set are not flagged. This is a prioritization tool for a subset of pages, not a comprehensive decline-detection system.

3. Predictive performance is moderate, not strong

ROC-AUC of 0.6655 indicates real, usable signal above chance, but it is far from the discrimination a reader might associate with a "solved" prediction problem. Individual predictions carry real uncertainty, and the model's own calibrated probabilities should be read as exactly that — probabilities, not verdicts.

4. Threshold choice is a trade-off, not a fact

The 0.60 threshold used throughout this report is one defensible choice among several (see Figure 3, §4). A team with more review capacity might reasonably choose 0.50 and accept lower precision for higher recall.

5. The baseline is a narrower comparison than it may first appear

As stated in §3, "baseline" here means an earlier version of the same model family, not an independent non-ML heuristic. The reported lift ($1.47\times$) is lift over random selection at the observed base rate — it is not evidence that this model beats the simplest rule a team might already be using informally.

6. No single feature drives the model

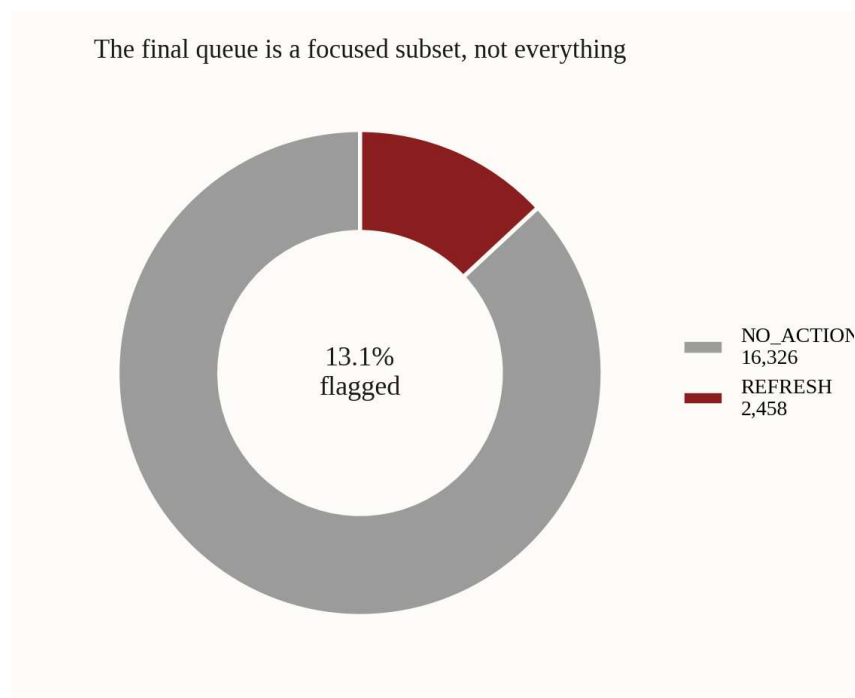
Feature importance is spread thin across the top ten inputs (Figure 4, §6), with the leading feature contributing barely above 0.01 mean importance. This is consistent with the model's moderate discrimination — there is no one dominant signal being exploited, for better or worse.

6 RANKED RECOMMENDATIONS

The action playbook

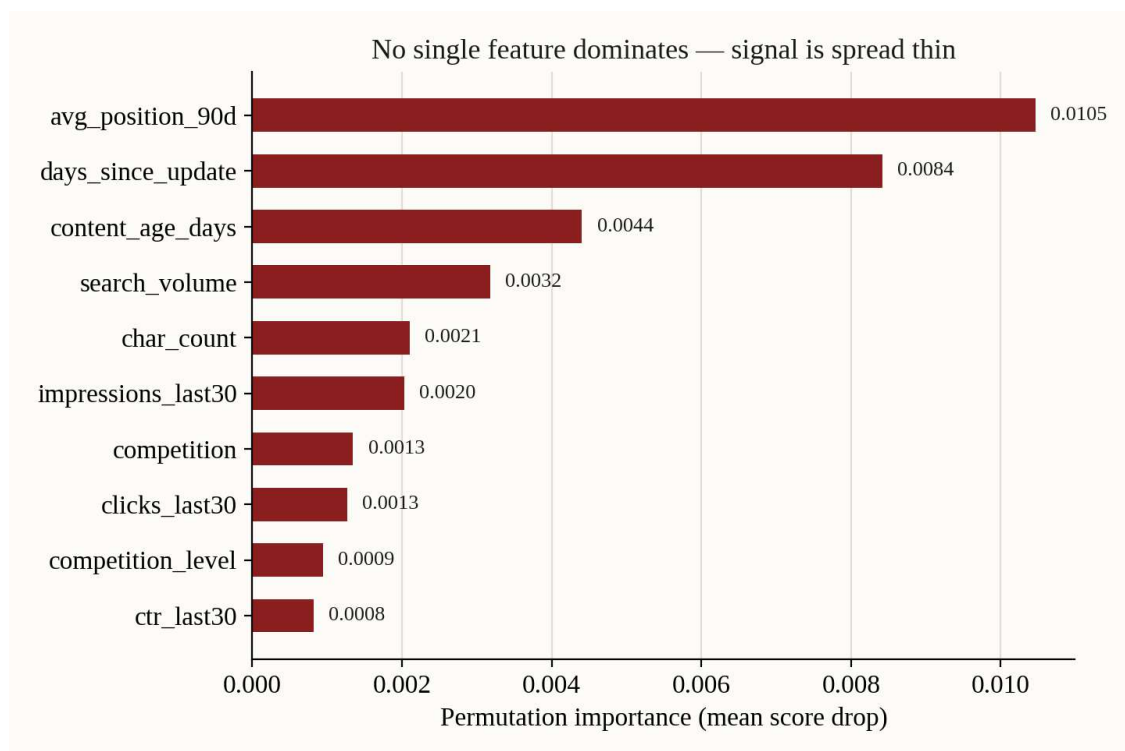
Every page in the portfolio receives a calibrated decline probability. Pages are ranked from highest to lowest, and a single operating rule converts the ranking into an action:

Condition	Action
Decline probability ≥ 0.60	REFRESH — added to the review queue
Decline probability < 0.60	NO_ACTION — no review triggered this cycle



Of 18,784 test pages: 2,458 REFRESH, 16,326 NO_ACTION.

Figure 5. A 13.1% flag rate — the queue is meant to be short enough that an editor will actually work through it.



Permutation importance, final 23-feature model.

Figure 6. Average position and staleness lead, but only barely — consistent with the model's moderate overall discrimination.

Table 2. Queue summary, test set

Metric	Value
Pages recommended (REFRESH)	2,458
Pages not flagged (NO_ACTION)	16,326
Recommendation rate	13.09%
Measured precision	59.28%
Measured recall	19.29%
Top-ranked page's probability	≈ 0.790

Base decline rate for this test set is 43.05% (\$4) — read the 59.28% precision above against that floor, not against zero.

How an editor should use this tomorrow

1. **Start at the top.** Review the highest-probability REFRESH pages first — the queue is already ranked by confidence.
2. **Diagnose before acting.** For each flagged page, check recent impressions and clicks, CTR movement, average position, content age, days since last update, and search-demand context before deciding what a refresh should even mean for that page.
3. **Treat NO_ACTION as "not this cycle," not "safe forever."** Re-score the full portfolio on a regular cadence rather than treating one pass as final.

Explicit no-go list:

- Do not treat a REFRESH flag as proof that a page's decline is caused by anything the model measured.

- Do not auto-publish content changes based on the score alone — the output is a review trigger, not an editorial decision.
- Do not apply this model to portfolios that look structurally different from the training data (different content types, languages, or verticals) without re-validating.
- Do not treat the 0.60 threshold as fixed — revisit it whenever review capacity changes.

Monitoring

Re-score the portfolio each time the warehouse refreshes, and re-check calibration (reliability tracked in the notebooks) whenever precision on newly-reviewed pages drifts noticeably from the 59.3% measured here.

7 REPRODUCIBILITY

Re-running this from a fresh clone

All code, notebooks, and saved model artifacts referenced in this report live in the linked repository under `work/notebooks/` and `work/outputs/`.

```
git clone <https://github.com/space-0d/flyrank >
cd flyrank
pip install -r requirements.txt # open work/notebooks/capstone.ipynb.
```

Random state was fixed at `random_state=42` across every model, split, and calibration step in this pipeline. The trained model, its preprocessor, and its feature list were serialized independently:

- `calibrated_improved_rf.pkl` — the final calibrated classifier
- `preprocessor_improved.pkl` — the fitted feature preprocessor
- `rf_improved.pkl` — the underlying uncalibrated Random Forest
- `feature_list.pkl` / `model_metadata.json` — the exact feature order and run metadata

The saved model was reloaded and re-scored against the same test set as a verification step: the maximum probability difference between the original and reloaded predictions was 0.000000000000, and the reloaded predictions matched the original recommendation set exactly. That check confirms the saved model artifact is faithful to the one that produced these numbers — it does not by itself confirm the full pipeline reproduces from a completely fresh clone and raw data. Anyone reviewing this work should still run the notebook top to bottom on a clean runtime before relying on these figures.

8 ACKNOWLEDGMENTS & DATA CREDIT

Built on the **FlyRank ML Internship dataset** — flyrank.ai